

Spring REST - POST, PUT, DELETE and Validation

RESTful Web Service resource naming guidelines - CountryController

CountryController.java - modified to follow REST URL conventions

```
package com.cognizant.springlearn.controller;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;

@RestController
@RequestMapping("/countries")
public class CountryController {

    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);

    @Autowired
    private CountryService countryService;

    @GetMapping
    public List<Country> getAllCountries() {
        LOGGER.info("START");
        List<Country> countryList = countryService.getAllCountries();
        LOGGER.info("END");
        return countryList;
    }

    @GetMapping("/{code}")
    public Country getCountry(@PathVariable String code) {
        LOGGER.info("START");
        Country country = countryService.getCountry(code);
        LOGGER.info("END");
        return country;
    }
}
```

Create RESTful Web Service to handle POST request of Country

CountryController.java - addCountry() added

```
@PostMapping
public void addCountry() {
    LOGGER.info("Start");
}
```

Output - curl -i -X POST -s /countries

```
HTTP/1.1 200
Content-Length: 0
Date: Tue, 21 Jul 2026 07:12:04 GMT
```

Output - console log

```
260721|12:42:04.117|http-nio-8090-exec-1|INFO |CountryController
addCountry|Start
```

Read country data as a bean in RESTful Web Service

CountryController.java - addCountry() updated

```
@PostMapping
public Country addCountry(@RequestBody Country country) {
    LOGGER.info("Start");
    LOGGER.debug("Country : {}", country);
    return country;
}
```

Output - curl -i -H 'Content-Type: application/json' -X POST -s -d '{"code":"IN","name":"India"}' /countries

```
HTTP/1.1 200
Content-Type: application/json;charset=UTF-8
Transfer-Encoding: chunked
Date: Tue, 21 Jul 2026 07:14:52 GMT

{"code":"IN","name":"India"}
```

Output - curl with misspelled attribute 'nae' instead of 'name'

```
HTTP/1.1 200
Content-Type: application/json;charset=UTF-8
Transfer-Encoding: chunked
Date: Tue, 21 Jul 2026 07:15:10 GMT

{"code":"IN","name":null}
```

Validating country code

Country.java - validation annotations added

```
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

public class Country {

    @NotNull
    @Size(min = 2, max = 2, message = "Country code should be 2 characters")
    private String code;

    private String name;

    // getters and setters
}
```

CountryController.java - addCountry() with manual validation

```
@PostMapping
public Country addCountry(@RequestBody Country country) {
    LOGGER.info("Start");

    ValidatorFactory factory = Validation.buildDefaultValidatorFactory();
    Validator validator = factory.getValidator();
    Set<ConstraintViolation<Country>> violations = validator.validate(country);

    List<String> errors = new ArrayList<String>();
    for (ConstraintViolation<Country> violation : violations) {
        errors.add(violation.getMessage());
    }
}
```

```

        if (violations.size() > 0) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, errors.toString());
        }

        LOGGER.debug("Country : {}", country);
        return country;
    }

```

Output - curl with single character country code

```

HTTP/1.1 400
Content-Type: application/json;charset=UTF-8
Transfer-Encoding: chunked
Date: Tue, 21 Jul 2026 07:19:36 GMT
Connection: close

{"timestamp":"2026-07-21T07:19:36.506+0000","status":400,"error":"Bad Request","message":"[Country code should be 2 characters]","path":"/countries"}

```

Include global exception handler for validation errors

GlobalExceptionHandler.java

```

package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

@ControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @Override
    protected ResponseEntity<Object>
    handleMethodArgumentNotValid(MethodArgumentNotValidException ex,
        HttpHeaders headers, HttpStatus status, WebRequest request) {
        LOGGER.info("Start");
        return null;
    }
}

```

CountryController.java - addCountry() using @Valid, manual validation code removed

```

@PostMapping
public Country addCountry(@RequestBody @Valid Country country) {
    LOGGER.info("Start");
    LOGGER.debug("Country : {}", country);
    return country;
}

```

Output - curl with single character country code

```

260721|12:26:41.310|http-nio-8090-exec-3|INFO |GlobalExceptionHandler
|handleMethodArgumentNotValid|Start

```

Note observed: CountryController log for Start is not present in the console, only GlobalExceptionHandler's Start log shows up, which confirms the controller method is never invoked when validation fails.

Response with bad request in global exception handler

GlobalExceptionHandler.java - handleMethodArgumentNotValid() completed

```
@Override
protected ResponseEntity<Object>
handleMethodArgumentNotValid(MethodArgumentNotValidException ex,
    HttpHeaders headers, HttpStatus status, WebRequest request) {

    LOGGER.info("Start");

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("timestamp", new Date());
    body.put("status", status.value());

    List<String> errors = ex.getBindingResult().getFieldErrors().stream()
        .map(x -> x.getDefaultMessage())
        .collect(Collectors.toList());

    body.put("errors", errors);
    LOGGER.info("End");

    return new ResponseEntity<>(body, headers, status);
}
```

Output - curl with single character country code

```
HTTP/1.1 400
Content-Type: application/json;charset=UTF-8
Transfer-Encoding: chunked
Date: Tue, 21 Jul 2026 07:26:41 GMT
Connection: close

{"timestamp":"2026-07-21T07:26:41.277+0000","status":400,"errors":["Country code should be 2 characters"]}
```

Implement REST service for updating an employee

Employee.java - validations added

```
import java.util.Date;
import java.util.List;
import javax.validation.constraints.Min;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;
import com.fasterxml.jackson.annotation.JsonFormat;

public class Employee {

    @NotNull
    private Integer id;

    @NotNull
    @NotBlank
    @Size(min = 1, max = 30)
    private String name;
```

```

    @NotNull
    @Min(0)
    private Double salary;

    @NotNull
    private Boolean permanent;

    @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "dd/MM/yyyy")
    private Date dateOfBirth;

    private Department department;

    private List<Skill> skillList;

    // getters and setters
}

```

Department.java - validations added

```

import javax.validation.constraints.NotBlank;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

public class Department {

    @NotNull
    private Integer id;

    @NotNull
    @NotBlank
    @Size(min = 1, max = 30)
    private String name;

    // getters and setters
}

```

Skill.java - validations added

```

import javax.validation.constraints.NotBlank;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

public class Skill {

    @NotNull
    private Integer id;

    @NotNull
    @NotBlank
    @Size(min = 1, max = 30)
    private String name;

    // getters and setters
}

```

EmployeeNotFoundException.java

```

package com.cognizant.springlearn.service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

```

```

@ResponseStatus(value = HttpStatus.NOT_FOUND, reason = "Employee not found")
public class EmployeeNotFoundException extends RuntimeException {

    private static final long serialVersionUID = 1L;

}

```

EmployeeDao.java - updateEmployee() added

```

public void updateEmployee(Employee employee) {
    LOGGER.info("START");

    boolean found = false;
    for (int i = 0; i < EMPLOYEE_LIST.size(); i++) {
        if (EMPLOYEE_LIST.get(i).getId() == employee.getId()) {
            EMPLOYEE_LIST.set(i, employee);
            found = true;
            break;
        }
    }

    if (!found) {
        throw new EmployeeNotFoundException();
    }

    LOGGER.info("END");
}

```

EmployeeService.java - updateEmployee() added

```

@Transactional
public void updateEmployee(Employee employee) {
    LOGGER.info("START");
    employeeDao.updateEmployee(employee);
    LOGGER.info("END");
}

```

EmployeeController.java - updateEmployee() added

```

@PutMapping
public void updateEmployee(@RequestBody @Valid Employee employee) throws
EmployeeNotFoundException {
    LOGGER.info("START");
    employeeService.updateEmployee(employee);
    LOGGER.info("END");
}

```

GlobalExceptionHandler.java - handleHttpMessageNotReadable() added

```

@Override
protected ResponseEntity<Object>
handleHttpMessageNotReadable(HttpMessageNotReadableException ex,
    HttpHeaders headers, HttpStatus status, WebRequest request) {

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("timestamp", new Date());
    body.put("status", status.value());
    body.put("error", "Bad Request");

    List<String> errors = new ArrayList<String>();
    if (ex.getCause() instanceof InvalidFormatException) {
        final Throwable cause = ex.getCause() == null ? ex : ex.getCause();
        for (InvalidFormatException.Reference reference : ((InvalidFormatException)
cause).getPath()) {

```

```

        body.put("message", "Incorrect format for field '" + reference.getFieldName() +
        """);
    }
}

return new ResponseEntity<>(body, headers, status);
}

```

Output - Postman PUT /employees

Request body:

```

{
  "id": 2,
  "name": "Divya Menon",
  "salary": 55000,
  "permanent": true,
  "dateOfBirth": "14/03/1994",
  "department": { "id": 1, "name": "IT" },
  "skillList": [ { "id": 2, "name": "Angular" }, { "id": 3, "name": "SQL" } ]
}

```

Response: 200 OK (empty body)

Output - Postman GET /employees, verifying the update

```

[
  { "id": 1, "name": "Arun Kumar", "salary": 45000.0, "permanent": true, ... },
  { "id": 2, "name": "Divya Menon", "salary": 55000.0, "permanent": true,
    "skillList": [ { "id": 2, "name": "Angular" }, { "id": 3, "name": "SQL" } ] },
  { "id": 3, "name": "Karthik Raja", "salary": 38000.0, "permanent": false, ... },
  { "id": 4, "name": "Priya Sundaram", "salary": 61000.0, "permanent": true, ... }
]

```

SpringLearnApplicationTests.java - MockMvc test for the exceptional scenario

```

@Test
public void testUpdateEmployeeNotFound() throws Exception {
    String payload = "{\"id\":99,\"name\":\"Ghost Employee\", \"salary\":10000, \"permanent\":true}";

    ResultActions actions = mvc.perform(put("/employees")
        .contentType(MediaType.APPLICATION_JSON)
        .content(payload));

    actions.andExpect(status().isNotFound());
    actions.andExpect(status().reason("Employee not found"));
}

```

Output - run as JUnit Test

```

260721|12:58:14.221|main          |INFO |EmployeeController |
updateEmployee|START
260721|12:58:14.223|main          |INFO |EmployeeService    |
updateEmployee|START
260721|12:58:14.224|main          |INFO |EmployeeDao        |
updateEmployee|START

JUnit
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0

```

Implement REST DELETE Service

EmployeeDao.java - deleteEmployee() added

```

public void deleteEmployee(int id) {
    LOGGER.info("START");

    boolean removed = EMPLOYEE_LIST.removeIf(e -> e.getId() == id);

    if (!removed) {
        throw new EmployeeNotFoundException();
    }

    LOGGER.info("END");
}

```

EmployeeService.java - deleteEmployee() added

```

@Transactional
public void deleteEmployee(int id) {
    LOGGER.info("START");
    employeeDao.deleteEmployee(id);
    LOGGER.info("END");
}

```

EmployeeController.java - deleteEmployee() added

```

@DeleteMapping("/{id}")
public void deleteEmployee(@PathVariable int id) throws EmployeeNotFoundException {
    LOGGER.info("START");
    employeeService.deleteEmployee(id);
    LOGGER.info("END");
}

```

Output - Postman DELETE /employees/3

Response: 200 OK (empty body)

Output - Postman GET /employees, verifying deletion

```

[
  { "id": 1, "name": "Arun Kumar", ... },
  { "id": 2, "name": "Divya Menon", ... },
  { "id": 4, "name": "Priya Sundaram", ... }
]

```

Output - Postman DELETE with an id that does not exist, /employees/99

```

HTTP/1.1 404
Content-Type: application/json; charset=UTF-8

{
  "timestamp": "2026-07-21T13:04:52.611+0000",
  "status": 404,
  "error": "Not Found",
  "message": "Employee not found",
  "path": "/employees/99"
}

```